Progetto di Reti Logiche

Nicolò Spuri[1] and Ayoub Salim[2]

[1]Codice Persona: 10855846, Matricola: 244611
[2]Codice Persona: 10839480, Matricola: 216173

Giugno 2025

POLITECNICO
MILANO 1863

Indice

1 Introduzione **3**
1.1 Obiettivo . . . 3
1.2 Specifiche generali . . . 3
1.3 Protocollo comunicazione . . . 4
1.4 Codifica (Filtro Differenziale) . . . 5
1.5 Interfaccia del modulo . . . 6

2 Architettura **7**
2.1 Segnali Principali . . . 7
2.2 Variabili interne . . . 8
2.3 Descrizione Processi . . . 8
2.3.1 Processo Cambio di Stato . . . 8
2.3.2 Processo Logica Stato Prossimo . . . 8
2.3.3 Processo Comportamentale Principale . . . 8
2.4 Descrizione Stati (FSM) . . . 11
2.4.1 IDLE . . . 11
2.4.2 READ_K1 . . . 11
2.4.3 READ_K2 . . . 11
2.4.4 READ_S . . . 11
2.4.5 READ_COEFFS . . . 11
2.4.6 READ_DATA . . . 11
2.4.7 PROCESS_DATA . . . 11
2.4.8 WRITE_RESULTS . . . 12
2.4.9 DONE_STATE . . . 12

3 Risultati Sperimentali **13**
3.1 Report Di Sintesi . . . 13
3.2 Simulazioni . . . 14
3.2.1 tb2425 . . . 14
3.2.2 tbMaxResult . . . 14
3.2.3 tbMinResult . . . 15
3.2.4 tbNoReset . . . 15
3.2.5 tbConReset . . . 16
3.2.6 tbKMax . . . 17
3.2.7 Altri test . . . 18

4 Conclusioni **19**

1 Introduzione

1.1 Obiettivo

Lo scopo del progetto è implementare un modulo hardware descritto in VHDL che possa interfacciarsi con una memoria esterna. Il modulo è progettato per leggere una sequenza di dati numerici W e i parametri di configurazione dalla memoria (lunghezza della sequenza, tipo di filtro, coefficienti), applicare alla sequenza uno dei due filtri differenziali, e infine scrivere la sequenza elaborata R nuovamente sulla memoria.

1.2 Specifiche generali

Il modulo interagisce con una memoria da cui preleva sia i parametri che la sequenza di dati da elaborare e sulla quale scriverà il risultato finale. I dati sulla memoria sono organizzati in modo sequenziale partendo da un indirizzo base ADD fornito in input.

Parametri Iniziali (17 byte):

- All'indirizzo `ADD` e `ADD`+1: Due byte (`K1`, `K2`) che indicano la lunghezza totale K (in byte, K $\geq$ 7) della sequenza W da elaborare. K1 è il byte più significativo.
- All'indirizzo `ADD`+2 abbiamo un byte S, il cui bit meno significativo (S(0)) seleziona l'ordine del filtro differenziale da applicare: se S(0) = 0 -> ordine 3, S(0) = 1 -> ordine 5.
- Infine dall'indirizzo `ADD`+3 a `ADD`+16 abbiamo 14 byte (C1-C14) che memorizzano i coefficienti dei due filtri. Il filtro di ordine 3 utilizza i coefficienti C1-C7, mentre il filtro di ordine 5 utilizza C8-C14.

Sequenza Dati di Input (W):

- Dall'indirizzo `ADD`+17 a `ADD`+17+`K`-1: La sequenza `W` di `K` byte da elaborare. Ogni byte rappresenta un valore numerico in complemento a 2, compreso nell'intervallo $[-128, +127]$.

Sequenza Dati di Output (R):

- Il modulo scriverà la sequenza elaborata `R`, anch'essa di `K` byte, negli indirizzi di memoria a partire da `ADD`+17+`K` fino a `ADD`+17+`K`+`K`-1. I valori calcolati per `R` che eccedono l'intervallo $[-128, +127]$ vengono invece saturati ai limiti dell'intervallo stesso.

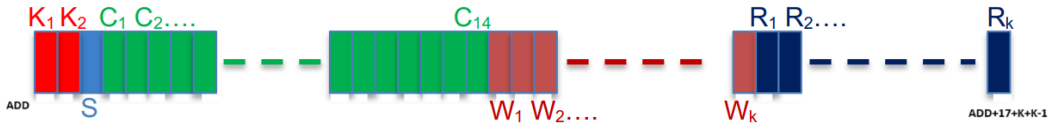


Figura 1: Indirizzi di memoria

1.3 Protocollo comunicazione

La comunicazione tra il modulo e l'ambiente esterno (simulato dal Test Bench) avviene secondo un protocollo specifico

1. Reset Iniziale: Prima di avviare la prima elaborazione, il modulo riceve sempre un segnale di reset asincrono (`i_rst` = '1').
2. Avvio Elaborazione: L'elaborazione inizia quando il segnale `i_start` viene portato a '1'. Contemporaneamente, sull'ingresso `i_add` viene fornito l'indirizzo base `ADD` in memoria.
3. Fase Attiva: Il segnale `i_start` deve rimanere alto per tutta la durata dell'elaborazione.
4. Fine Elaborazione: Una volta completata l'elaborazione e dopo aver scritto l'intera sequenza `R` in memoria, il modulo segnala la fine alzando il segnale `o_done` (portandolo a '1').
5. Attesa Riconoscimento: Il segnale `o_done` rimane alto finché il segnale `i_start` non viene riportato a '0'.
6. Pronto per Nuova Elaborazione: Il modulo è pronto per una nuova elaborazione solo dopo che `i_start` è stato riportato a '0'. Un nuovo ciclo può iniziare alzando nuovamente `i_start`, senza dover attendere un reset.

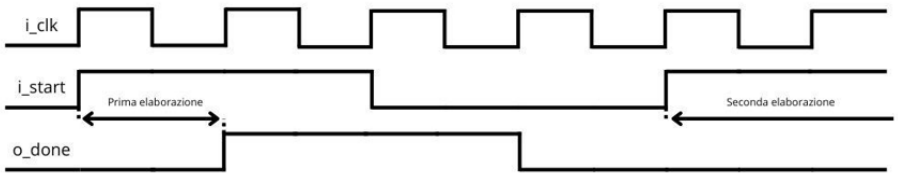


Figura 2: Protocollo di comunicazione START/DONE

1.4 Codifica (Filtro Differenziale)

Il cuore dell'elaborazione consiste nell'applicazione di un filtro differenziale alla sequenza `W`. La funzione matematica del filtro è data da:

$$f'(i) = \frac{1}{n} \sum_{j=-l}^{+l} C_j f[j+i]$$

dove:

- $f'(i)$ è l'i-esimo campione della sequenza filtrata (prima della saturazione).
- C_j sono i coefficienti del filtro selezionato tramite il byte `S`.
- $f[j+i]$ sono i campioni della sequenza di input `W`. L'indice j varia da $-l$ a $+l$, dove $l = 2$ per il filtro di ordine 3 e $l = 3$ per il filtro di ordine 5.

Per calcolare i valori $f'(i)$ all'inizio e alla fine della sequenza `W` (dove l'indice $j+i$ cadrebbe fuori dai limiti da 0 a `K`-1), si utilizzano valori pari a 0 (padding).

n è il fattore di normalizzazione, specifico per ciascun filtro ($n = 12$ per ordine 3, $n = 60$ per ordine 5).

La normalizzazione (divisione per n) non viene eseguita con una divisione diretta, ma è approssimata tramite somme di shift aritmetici a destra, come indicato nelle specifiche:

$$\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$$
$$\frac{1}{60} \approx \frac{1}{64} + \frac{1}{1024}$$

Per mitigare l'errore di troncamento introdotto dagli shift a destra su numeri negativi, viene applicata una correzione: se il valore intermedio $\sum C_j f[j+i]$ è negativo, si aggiunge 1 al risultato di ciascun singolo shift utilizzato nell'approssimazione della divisione.

Il valore finale `R`(i) memorizzato è il risultato normalizzato $f'(i)$ saturato nell'intervallo $[-128, +127]$.

1.5 Interfaccia del modulo

L'interfaccia VHDL del modulo è la seguente:

- `i_clk`: (`in std_logic`) segnale di clock, tutte le operazioni sincrone avvengono sul fronte di salita.
- `i_rst`: (`in std_logic`) segnale di reset asincrono che resetta lo stato interno del modulo.
- `i_start`: (`input std_logic`) segnale di avvio/controllo elaborazione.
- `i_add`: (`in std_logic_vector(15 downto 0)`) segnale ADD generato dal Test-Bench e rappresenta il punto da cui parte la sequenza che elaboriamo.
- `o_done`: (`out std_logic`) segnale di fine elaborazione.
- `o_mem_addr`: (`out std_logic_vector(15 downto 0)`) indirizzo inviato alla memoria per le operazioni di lettura o scrittura.
- `i_mem_data`: (`in std_logic_vector(7 downto 0)`) dato a 8 bit letto dalla memoria all'indirizzo `o_mem_addr`.
- `o_mem_data`: (`out std_logic_vector(7 downto 0)`) dato a 8 bit da scrivere in memoria all'indirizzo `o_mem_addr`.
- `o_mem_we`: (`out std_logic`) segnale di Write Enable per la memoria, esso vale 1 in scrittura o 0 in lettura.
- `o_mem_en`: (`out std_logic`) segnale di Enable per la memoria, esso deve essere messo a 1 per poter comunicare sia in lettura che in scrittura.

2 Architettura

L'architettura del nostro modulo è stata implementata utilizzando un approccio Behavioral all'interno di una sola architecture VHDL, con un'architettura logica di controllo implementata tramite una FSM.

2.1 Segnali Principali

Per gestire il flusso di dati e il controllo della FSM e dei suoi stati abbiamo definito dei segnali interni:

- `current_state`, `next_state`: segnali di tipo state_type che memorizzano lo stato corrente e quello futuro della FSM.
- `current_read_address`, `current_write_address`: segnali unsigned(15 downto 0) utilizzati per tenere traccia degli indirizzi di lettura e scrittura correnti nella memoria esterna.
- `k_length`: segnale unsigned(15 downto 0) che memorizza la lunghezza K della sequenza di dati W, letta dai primi due byte all'inizio.
- `filter_order`: segnale std_logic che memorizza il bit meno significativo del byte S e che indica quale filtro usare ('0' per ordine 3, '1' per ordine 5).
- `filter_coeffs`: array di tipo signed(7 downto 0) per memorizzare i 7 coefficienti del filtro selezionato.
- `input_data`: array di tipo signed(7 downto 0) che fa da buffer per memorizzare i campioni della sequenza W necessari per il calcolo del filtro in un determinato momento.
- `output_data`: segnale signed(7 downto 0) che contiene il risultato del filtro (normalizzato e saturato) pronto per essere scritto in memoria.
- `coeff_counter`, `data_counter`: segnali integer usati come contatori per i cicli di lettura di coefficienti e dati.
- `idle_wait`, `read_wait`, `write_done`: segnali std_logic usati come flag per gestire la sincronizzazione e l'ordine delle operazioni (tipo attesa tra accessi alla memoria).

2.2 Variabili interne

Oltre ai segnali abbiamo sfruttato le seguenti variabili. La scelta di utilizzare variabili da 24 bit è data dalla grandezza del numero massimo ottenibile, dato dal prodotto tra i coefficienti dei filtri e dei 7 dati di `input_data`, esprimibile con 18 bit, per cui il multiplo di 8 più vicino che ci permette di rappresentarlo è 24.

- `temp_result`: variabile signed(23 downto 0) utilizzata per svolgere le operazioni di calcolo applicando i coefficienti di input ai dati di `input_data`.
- `normalized_result`: variabile signed(23 downto 0) utilizzata invece per applicare la normalizzazione sul risultato ottenuto all'interno di `temp_result`. Infine viene applicata la saturazione a questo valore e viene salvato direttamente in `output_data`.

2.3 Descrizione Processi

L'intera logica è contenuta in tre processi principali:

2.3.1 Processo Cambio di Stato

Processo sincrono che serve ad aggiornare lo stato corrente `current_state` con il valore calcolato per lo stato successivo `next_state` ad ogni fronte di salita del clock i_clk. Oltre a questo gestisce il reset asincrono i_rst, forzando lo stato IDLE.

2.3.2 Processo Logica Stato Prossimo

Processo interamente combinatorio che serve a definire le transizioni tra gli stati della FSM, calcola quale dovrà essere il `next_state` basandosi su quello corrente (`current_state`) e sui segnali di input e interni (`i_start`, `idle_wait`, `i_add`, `data_counter`, `coeff_counter`, `k_length`, `read_wait`, `write_done`, `current_write_address`).

2.3.3 Processo Comportamentale Principale

In questo processo implementiamo la maggior parte del comportamento del modulo, agendo in base allo stato corrente (`current_state`) determinato dalla FSM. È un processo sincrono che si occupa di:

- Reset Iniziale: su reset (`i_rst` = 1), inizializza tutti i segnali interni, i contatori, gli array e le uscite ai valori di default.
- Controllo Memoria: Setta i segnali di controllo per la memoria (`o_mem_en`, `o_mem_we`, `o_mem_addr`) in base allo stato corrente e alle operazioni da svolgere (lettura parametri, lettura dati W, scrittura risultati R).

- Gestione Indirizzi e Contatori: Aggiorna i segnali di `current_read_address`, `current_write_address`, `coeff_counter`, `data_counter` man mano che l'elaborazione procede attraverso gli stati.
- Acquisizione Dati: Legge `i_mem_data` e memorizza i valori nei segnali appropriati (`k_length`, `filter_order`, `filter_coeffs`, `input_data`) negli stati corretti. Gestisce anche il padding a zero per i dati ai bordi della sequenza W.
- Elaborazione Dati: Nello stato `PROCESS_DATA`, esegue il calcolo della sommatoria $\sum C_j \cdot f[j+i]$, applica la normalizzazione approssimata con shift (e la correzione per i negativi) e infine satura il risultato per generare `output_data`. Utilizza variabili locali (`temp_result`, `normalized_result`) per i calcoli intermedi.
- Scrittura Risultati: Nello stato `WRITE_RESULTS`, inserisce in `o_mem_data` il valore calcolato e attiva `o_mem_we` per scrivere nella memoria.
- Controllo Output: Gestisce il segnale `o_done`.

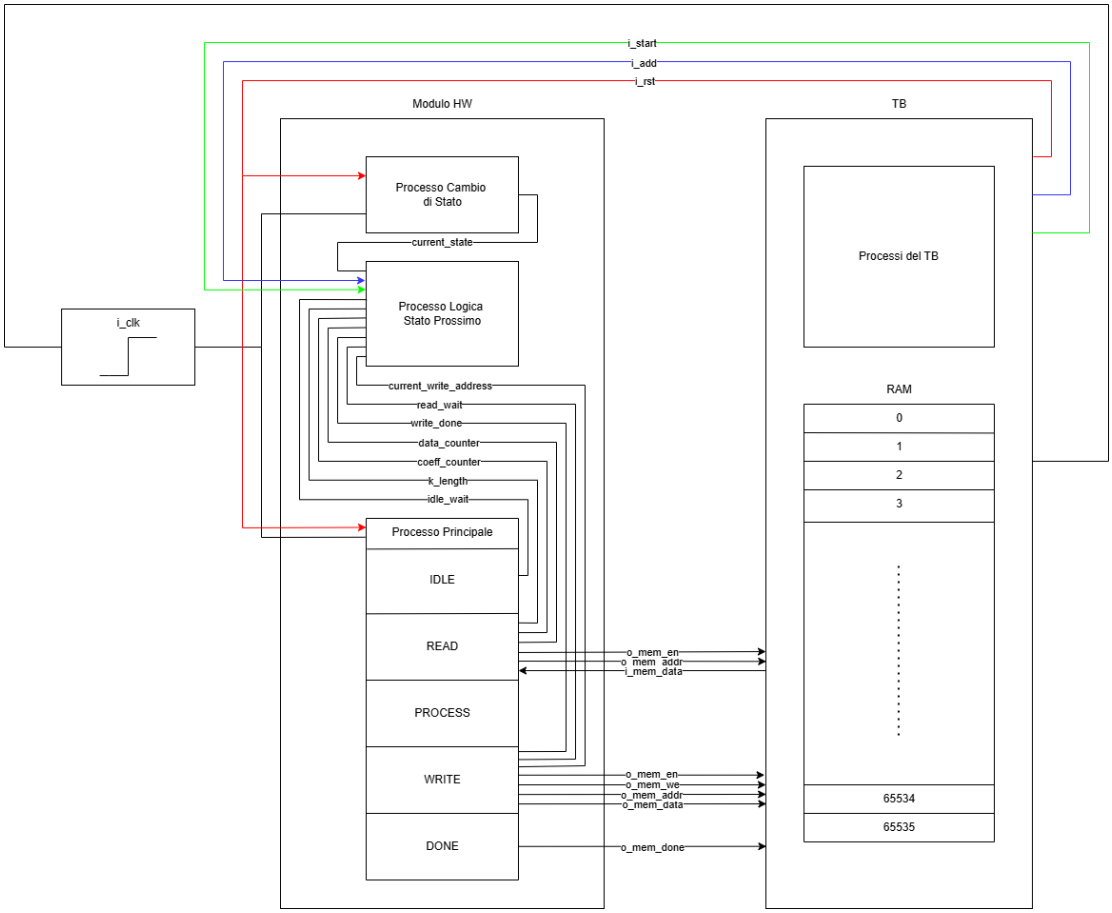


Figura 3: Architettura

2.4 Descrizione Stati (FSM)

2.4.1 IDLE

Stato di attesa iniziale, setta il segnale `o_done` a '0' (per permettere una successiva elaborazione) e aspetta che `i_start` diventi '1'. Quando arriva, legge l'indirizzo base `i_add` e prepara il primo accesso in memoria per leggere K1.

2.4.2 READ_K1

Legge il primo byte della lunghezza K e scorre l'indirizzo per la lettura del prossimo parametro.

2.4.3 READ_K2

Legge il secondo byte della lunghezza K e scorre l'indirizzo per la lettura del prossimo parametro.

2.4.4 READ_S

Legge il byte S che indica l'ordine del filtro, calcola l'indirizzo iniziale di scrittura per i risultati e scorre l'indirizzo per la lettura del prossimo parametro.

2.4.5 READ_COEFFS

Legge sequenzialmente i 14 byte dei coefficienti e memorizza solo i 7 coefficienti relativi al filtro selezionato (`filter_order`) nell'array `filter_coeffs`. Nella sua penultima esecuzione imposta l'indirizzo `current_read_address` a `i_add + 17 - 3` per la lettura della prima sequenza di dati da elaborare.

2.4.6 READ_DATA

Legge i 7 byte consecutivi (sequenza W), gestendo il padding a 0 per gli accessi fuori dai limiti della sequenza, e li memorizza nel buffer `input_data`. Nella sua ultima esecuzione, imposta l'indirizzo a `current_read_address - 6` per la lettura della prossima sequenza di dati.

2.4.7 PROCESS_DATA

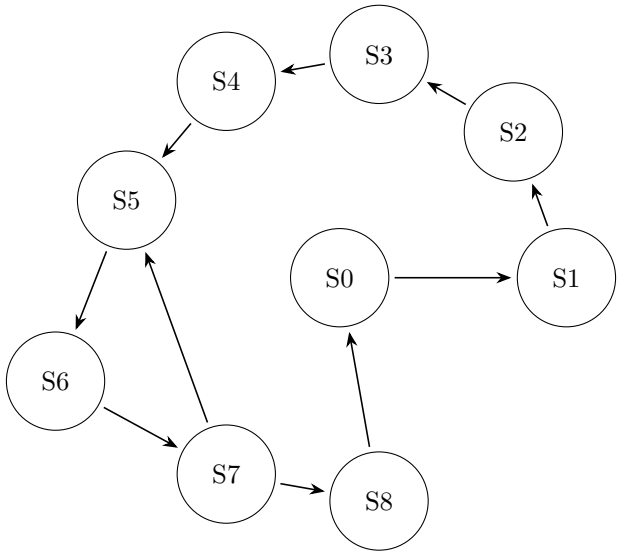
Esegue il calcolo del filtro utilizzando i coefficienti in `filter_coeffs` e i dati in `input_data`. Applica dunque normalizzazione e saturazione per ottenere `output_data`. In questo stato utilizziamo le variabili locali (`temp_result` e `normalized_result`) per i calcoli intermedi.

2.4.8 WRITE_RESULTS

Scrive il byte `output_data` calcolato all'indirizzo `current_write_address`, incrementa l'indirizzo di scrittura e prepara la lettura del prossimo set di 7 byte di input (se non si è arrivati alla fine della sequenza).

2.4.9 DONE_STATE

A questo punto l'elaborazione è terminata, quindi si alza il segnale `o_done` a 1 e si attende che `i_start` torni a 0 per poter ritornare allo stato `IDLE`.



S0: IDLE
S1: READ_K1
S2: READ_K2
S3: READ_S
S4: READ_COEFFS
S5: READ_DATA
S6: PROCESS_DATA
S7: WRITE_RESULTS
S8: DONE_STATE

Figura 4: Diagramma della Macchina a Stati Finiti (FSM)

3 Risultati Sperimentali

3.1 Report Di Sintesi

- Il Timing Summary della figura 3 mostra un `WNS` (Worst Negative Slack) di circa 4.8 ns, calcolato in funzione di un clock di periodo pari a 20 ns. Lo slack di 4.824 ns indica che il circuito rispetta i vincoli temporali imposti, con un percorso critico di circa 15.176 ns (20 ns - 4.824 ns).

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	4,824 ns	Worst Hold Slack (WHS):	0,178 ns	Worst Pulse Width Slack (WPWS):	9,600 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns	Total Pulse Width Negative Slack (TPWS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	339	Total Number of Endpoints:	339	Total Number of Endpoints:	211

All user specified timing constraints are met.

Figura 5: Timing Summary

- Il report sull'utilization mostra che il progetto non ha alcun latch, confermando la correttezza delle scelte progettuali e della logica di funzionamento. Inoltre l'utilizzo delle risorse è molto contenuto rispetto a quelle disponibili sul dispositivo target, dimostrando l'efficienza dell'implementazione realizzata.

Site Type	Used	Fixed	Available	Util%
Slice LUTs	945	0	41000	2.30
LUT as Logic	945	0	41000	2.30
LUT as Memory	0	0	13400	0.00
Slice Registers	210	0	82000	0.26
Register as Flip Flop	210	0	82000	0.26
Register as Latch	0	0	82000	0.00
F7 Muxes	16	0	20500	0.08
F8 Muxes	0	0	10250	0.00

Figura 6: Slice Logic

3.2 Simulazioni

Il modulo ha superato sia la simulazione *behavioral* che la *functional*, entrambe *pre* e *post-synthesis* utilizzando dei testbench per coprire sia delle condizioni standard e sia dei corner case più complessi. Tutte le simulazioni sono state documentate e hanno confermato il corretto funzionamento del modulo.

3.2.1 tb2425

Questo test utilizza un filtro differenziale di ordine 3. L'elaborazione dei dati e la gestione dei segnali di controllo sono risultate corrette e in linea con le specifiche richieste.

3.2.2 tbMaxResult

In questo test abbiamo impiegato un filtro di ordine 5 per testare il caso limite in cui l'applicazione del filtro produce il valore massimo ottenibile prima della fase di normalizzazione.

- Gli input (`i_mem_data`) sono tutti impostati al valore minimo negativo (-128).
- I coefficienti del filtro di ordine 5 sono tutti impostati al valore minimo negativo (-128).

La simulazione conferma la corretta gestione del valore massimo e del troncamento.

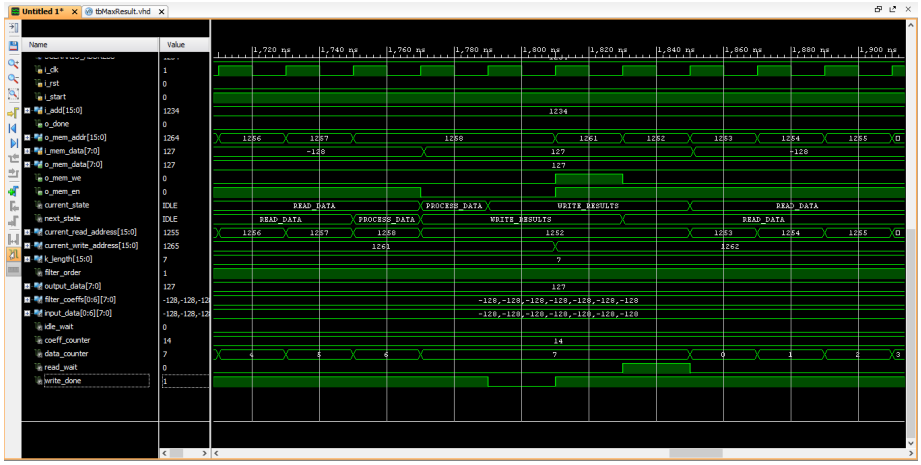


Figura 7: Max

3.2.3 tbMinResult

Sulla falsa riga del testbench precedente, con questo abbiamo testato il caso limite in cui l'applicazione del filtro produce il valore minimo ottenibile prima della fase di normalizzazione. coefficienti sono rimasti a '–128', mentre i dati di input sono stati portati a '127' per testare il comportamento ai valori minimi. Il modulo ha gestito correttamente i bit intermedi, evitando overflow e producendo un troncamento accurato.

- Gli input (`i_mem_data`) sono tutti impostati al valore massimo positivo (127).
- I coefficienti del filtro di ordine 5 sono tutti impostati al valore minimo negativo (-128).

La simulazione conferma la corretta gestione del valore minimo e del troncamento

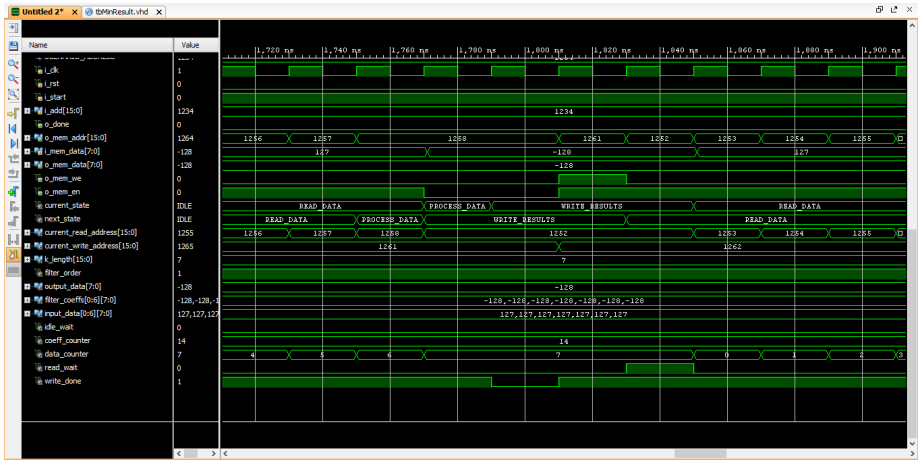


Figura 8: Min

3.2.4 tbNoReset

Questa simulazione esegue due elaborazioni consecutive senza reset tra le due, inserendo però un reset durante l'esecuzione della prima.

- **Primo Scenario:** Viene utilizzato un filtro di ordine 3 con il primo e l'ultimo coefficiente diversi da '0'. Dopo la lettura dei parametri viene alzato il segnale di reset, successivamente l'elaborazione procede senza interruzioni
- **Secondo Scenario:** La seconda elaborazione viene eseguita senza un reset del modulo e viene utilizzato un filtro di ordine 5.

La simulazione evidenzia la capacità del modulo di gestire un reset durante l'elaborazione e di eseguire scenari multipli in sequenza usando correttamente entrambi i filtri.

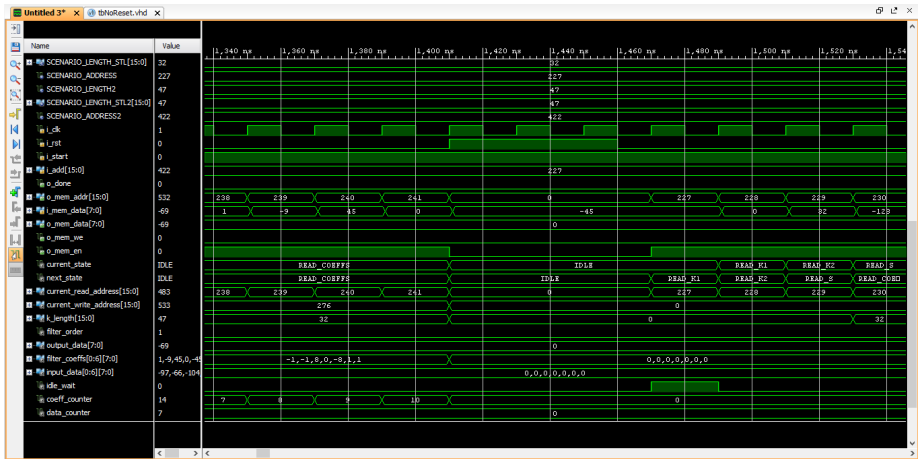


Figura 9: No Reset

3.2.5 tbConReset

In questo test due elaborazioni consecutive sono state intervallate da un reset esplicito prima dell'avvio della seconda. Il modulo ha ripristinato correttamente lo stato iniziale e ha continuato l'elaborazione correttamente.

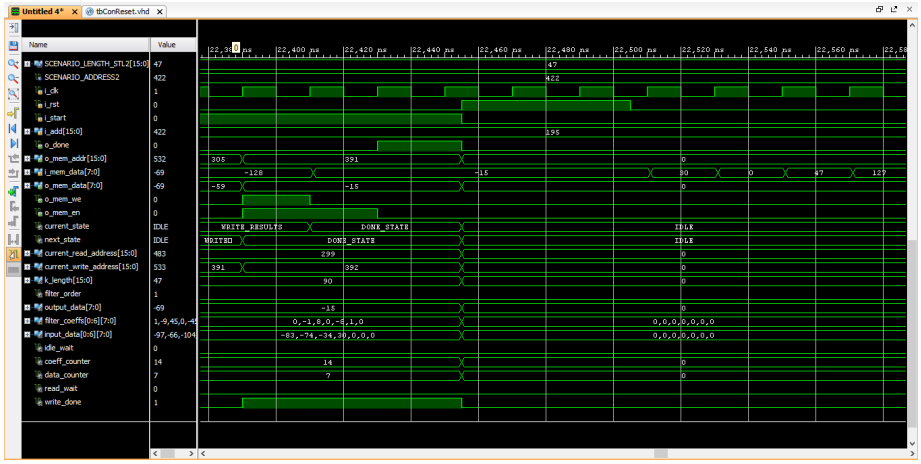


Figura 10: Con Reset

3.2.6 tbKMax

Questa simulazione testa il modulo con il valore massimo possibile per K (32759). Considerando i 65536 indirizzi di memoria disponibili e i 17 occupati dai parametri, rimane disponibile un numero dispari di indirizzi, considerando che metà vengono utilizzati in lettura e metà in scrittura, resta 1 byte libero e nonostante ciò l'abbiamo ritenuto un caso limite, dato che rappresenta la sequenza elaborabile più lunga possibile. La simulazione dimostra la capacità del modulo di gestire correttamente sequenze molto lunghe senza superare i limiti di memoria

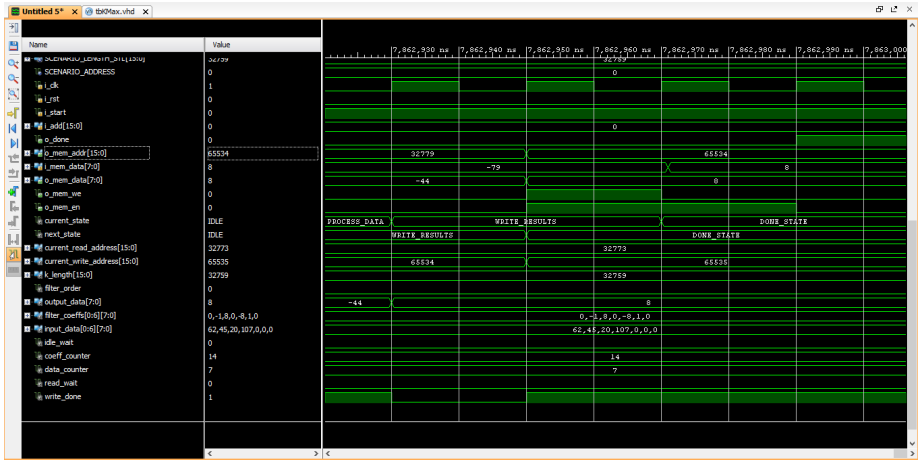


Figura 11: K Max

3.2.7 Altri test

Oltre ai test principali descritti sopra, abbiamo eseguito varie altre simulazioni, provando più elaborazioni consecutive, diversi valori di K, con e senza reset tra le elaborazioni, e cambiando le configurazioni operative del modulo. Questi test hanno permesso di esplorare scenari differenti e di confermare la stabilità generale del sistema. Non sono stati riportati nel dettaglio perché, pur confermando il corretto funzionamento del modulo, non rappresentano corner case significativi.

4 Conclusioni

Il modulo sintetizzato è in grado di applicare un filtro differenziale a una sequenza di dati memorizzati in memoria, e ha superato con successo tutti i test, sia in Behavioral che in Functional, entrambi Pre e Post-Synthesis. Ha dimostrato di essere funzionalmente corretto, gestendo correttamente casi limite come reset durante l'elaborazione, valori di input che causano saturazione ai limiti e sequenze di lunghezza massima (K=32759).

Durante lo sviluppo, l'ottimizzazione dell'architettura è stata fondamentale per migliorare le prestazioni. Inizialmente, avevamo optato per un buffer ampio per accumulare tutti i dati da elaborare prima dell'elaborazione, ma le simulazioni hanno mostrato che ciò causava importanti latenze. Per cui abbiamo adottato un approccio basato sull'elaborazione sequenziale di blocchi di 7 byte alla volta: il modulo legge un blocco dalla memoria, applica il filtro, scrive il risultato e ripete il processo.

Questa riduzione della dimensione del blocco ha migliorato significativamente la velocità di esecuzione, ottimizzando l'uso delle risorse hardware. Inoltre, abbiamo implementato più stadi per la lettura dei parametri, migliorando la chiarezza rispetto a un unico stadio, ad esempio `READ_PARAMETERS`.

In conclusione, il progetto ha soddisfatto tutti gli obiettivi prefissati risultando in un modulo VHDL funzionante e ottimizzato, trovando così un buon equilibrio tra specifiche e prestazioni.